

# **Curso Inicial Front-End**

## **Clase 16: Objetos**

## **Literales y Métodos de**

## **Array Avanzados**

Módulo 2: Dominio de JavaScript y Lógica de Programación



## Presentación

---

Un usuario no es solo un nombre; es un conjunto de datos (nombre, edad, email). En esta clase aprenderemos a usar **Objetos**, la estructura fundamental de **JavaScript** para representar entidades del mundo real. Dominaremos el uso de propiedades y métodos, y conoceremos las herramientas estáticas de `Object` para manipular información compleja como un profesional.



## Objetivos

---

- Dominar la creación y el acceso a datos mediante la notación de punto y corchetes.
- Implementar métodos estáticos (`Object.keys`, `values`, `entries`) para iterar sobre objetos.
- Comprender la desestructuración de objetos para escribir código más limpio y moderno.



## Bloques Temáticos

---

- Tema 1: Estructuras de objetos y palabra reservada this.
  - ¿Qué es un Objeto Literal? El Carnet de Identidad
  - Acceso y Modificación: ¿Cómo leo el DNI?
  - Destructuring (Desestructuración de Objetos)
  - Referencia vs Valor: El Terror de las Entrevistas Técnicas
- Tema 2: Métodos funcionales de arrays (sin callbacks complejos).

En la clase anterior aprendimos a guardar listas de datos sueltos en Arrays. Pero en la vida real, la información es compleja. Un "Usuario" no es solo el texto "Juan". Un usuario tiene un correo, una edad, una contraseña y permisos.

Si intentamos guardar un auto usando variables sueltas (`let colorAuto = "Rojo", let marcaAuto = "Ford"`), el código se vuelve un caos. Para resolver esto, **JavaScript** utiliza **Objetos Literales**: una súper-variable que agrupa datos relacionados bajo un mismo techo.



**Recurso Oficial:** [MDN Web Docs - Trabajar con Objetos](#)

# Tema 1: Estructuras de objetos y palabra reservada this.

## 1. ¿Qué es un Objeto Literal? El Carnet de Identidad

Imagina que un Objeto es como un **Carnet de Identidad (DNI)**. Es un documento único que contiene múltiples campos de información sobre una misma entidad.

Técnicamente, un objeto es una estructura de datos que guarda la información en pares de **Clave: Valor**.

- **Clave (Key):** Es el nombre de la etiqueta (ej: `edad`).
- **Valor (Value):** Es el dato real guardado allí (ej: `28`).

Un objeto tiene dos tipos de contenido:

1. **Propiedades (¿Cómo ES?):** Son características pasivas (textos, números, booleanos).
2. **Métodos (¿Qué HACE?):** Son funciones matemáticas o lógicas que viven adentro del objeto.

```
/* Usamos llaves { } para indicarle a JS que estamos creando un Objeto Literal */
const perfilUsuario = {
  // Clave : Valor
  nombre: "Lucas",      // Propiedad (String)
  edad: 28,             // Propiedad (Number)
```

```
esPremium: true,          // Propiedad (Boolean)

// Método: Una acción que el objeto puede realizar
saludar() {
  console.log(`Hola, mi nombre es ${this.nombre}`);
}
};
```

### ⚠ IMPORTANTE

**La Trampa de las Arrow Functions**  **NUNCA** uses una Arrow Function (`=>`) para crear un Método dentro de un objeto. Las flechas **no tienen identidad propia (no entienden la palabra `this`)**, sino que buscan su identidad en el archivo global. Si las usas como métodos, tu código no sabrá de qué "auto" o "usuario" estás hablando. Usa siempre la forma clásica: `saludar() { ... }`.

## 2. Acceso y Modificación: ¿Cómo leo el DNI?

Para leer un dato específico del objeto, tenemos dos herramientas. La elección de la herramienta correcta separa a los novatos de los profesionales.

### Comparativa: Punto vs Corchetes

| Herramienta           | Sintaxis Visual                      | ¿Cuándo la uso?                                             | Limitación fatal                                                    |
|-----------------------|--------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------|
| Notación de Punto     | <code>perfilUsuario.nombre</code>    | El 90% de las veces.<br>Es limpia, directa y fácil de leer. | ✗ Falla si quieres usar una Variable dinámica para buscar la clave. |
| Notación de Corchetes | <code>perfilUsuario["nombre"]</code> | Cuando la clave tiene espacios ( <code>"color</code>        | ⚠ Es más ruidosa visualmente.                                       |

| Herramienta | Sintaxis Visual | ¿Cuándo la uso?                              | Limitación fatal |
|-------------|-----------------|----------------------------------------------|------------------|
|             |                 | exterior") o cuando depende de una Variable. |                  |

### El Secreto del Dinamismo (Por qué amamos los corchetes):

```
/* El usuario elige en un menú desplegable qué dato quiere ver. Lo guardamos en una variable. */  
let eleccionDelUsuario = "edad";  
  
/* ❌ ERROR: JS buscará literalmente una propiedad que se llame "eleccionDelUsuario" y dará undefined. */  
console.log(perfilUsuario.eleccionDelUsuario);  
  
/* ✅ ÉXITO: JS primero lee la variable, descubre que dice "edad", y luego busca la clave "edad" en el objeto. */  
console.log(perfilUsuario[eleccionDelUsuario]);
```

## Optional Chaining (El Salvavidas: ?.)

Si le pides a una base de datos el código postal del usuario (`usuario.direccion.codigoPostal`), pero el usuario nunca llenó su dirección, **JS** lanzará un error crítico en rojo que romperá y detendrá toda tu página web (`Cannot read properties of undefined`).

**Solución Moderna:** Pon un signo de interrogación de seguridad.

`usuario.direccion?.codigoPostal` Esto le dice a **JS**: "Ve a buscar la dirección, pero si no existe, frena pacíficamente y devuélveme un simple `undefined` en lugar de destruir mi página".

## Métodos Maestros de Objetos

Para manipular objetos de forma profesional, **JavaScript** nos da herramientas estáticas a través de la palabra **Object**:

| Método                           | Función                                    | Resultado                            |
|----------------------------------|--------------------------------------------|--------------------------------------|
| <code>Object.keys(obj)</code>    | Extrae todas las claves.                   | Un Array de strings con los nombres. |
| <code>Object.values(obj)</code>  | Extrae todos los valores.                  | Un Array con los datos.              |
| <code>Object.entries(obj)</code> | Extrae pares <code>[clave, valor]</code> . | Un Array de Arrays (Matriz).         |
| <code>Object.assign(a, b)</code> | Copia propiedades de B en A.               | Un objeto combinado.                 |
| <code>Object.freeze(obj)</code>  | Congela el objeto.                         | El objeto se vuelve 100% inmutable.  |

Ejemplo de uso:

```
const usuario = { id: 1, nombre: "Ana", rol: "admin" };  
  
const claves = Object.keys(usuario); // ["id", "nombre", "rol"]  
const valores = Object.values(usuario); // [1, "Ana", "admin"]
```

## 3. Destructuring (Desestructuración de Objetos)

Es una técnica de sintaxis moderna inventada para que no tengas que escribir `perfilUsuario.nombre` veinte veces en un mismo archivo. Nos permite "extraer" copias de las propiedades y convertirlas en variables sueltas en una sola línea.

**Mapeo de Sintaxis:**

```
/* "Ve al objeto 'perfilUsuario', sácale una copia a 'nombre' y 'edad', y  
crea dos variables flotantes con esos mismos nombres" */  
const { nombre, edad } = perfilUsuario;  
  
console.log(nombre); // Imprime "Lucas" directamente, sin usar el punto.
```

## 4. Referencia vs Valor: El Terror de las Entrevistas Técnicas

Este es el concepto que más marea (y reprueba) a los desarrolladores Junior. En **JavaScript**, el signo igual (=) funciona de dos formas totalmente distintas según qué estés copiando.

- **Valores Primitivos (Textos, Números):** Funcionan como una **FOTO FÍSICA**. Si tú tienes la foto de un auto, vas a la fotocopidora y me das una copia (`let copia = original`), yo puedo pintar mi copia con marcador rojo, y tu foto original seguirá intacta.
- **Objetos y Arrays (Datos Complejos):** Funcionan como una **CARPETA COMPARTIDA DE GOOGLE DRIVE**. Cuando haces `const clon = autoOriginal`, NO estás copiando el auto. Estás entregando un enlace de acceso directo al mismo garaje. ¡Si tú le borras una puerta al `clon`, el `autoOriginal` también perderá la puerta instantáneamente!

🔴 **Checkpoint de Comprensión:** Tienes un objeto `const cuentaBancaria = { saldo: 1000 }`. Tu compañero hace `const miCuenta = cuentaBancaria`; y luego ejecuta `miCuenta.saldo = 0`. Sabiendo cómo funciona la Memoria de Referencia de los objetos, ¿cuánto dinero le quedó a `cuentaBancaria` y por qué?

## La Solución: El Spread Operator (...)



Para arreglar este problema y crear una fotocopia real (desconectada del Google Drive original), usamos los "tres puntos mágicos" (Spread Operator).  
Desparrama los datos viejos dentro de unas llaves nuevas:

```
/* Esto crea un objeto 100% nuevo y le inyecta las propiedades del viejo.  
¡Ahora son independientes! */  
const clonReal = { ...autoOriginal };
```

# Tema 2: Métodos funcionales de arrays (sin callbacks complejos)

---

Cuando trabajamos con **arrays**, muchas veces queremos buscar elementos, cortar partes de la lista o unir la información sin necesidad de escribir funciones o bucles **complejos**.

En la programación moderna se priorizan los métodos **no mutables** (que no modifican el array **original**, sino que te devuelven una **respuesta** nueva). Estos son los métodos esenciales que **no requieren callbacks** (funciones pasadas por parámetro) para funcionar.

## **.includes(elemento)**

Te dice con un simple **true** o **false** si un elemento exacto se encuentra dentro del array.

```
const invitados = ["Juan", "María", "Lucas"];

// Comprobación rápida:
console.log(invitados.includes("María")); // true
console.log(invitados.includes("Pedro")); // false
```

## **.indexOf(elemento)**

Busca un elemento de **izquierda a derecha** y te devuelve su **índice** (posición).  
**Si el elemento no existe en la lista, te devolverá -1.**

```
const podio = ["Sofía", "Leonel", "Mateo"];

// Buscar la posición:
console.log(podio.indexOf("Leonel")); // 1
console.log(podio.indexOf("Carlos")); // -1 (no está en el podio)
```

## .join(separador)

Toma todos los elementos del array y los une en una sola cadena de texto (String), **separándolos** por el texto o símbolo que tú elijas entre los paréntesis.

```
const ingredientes = ["Pan", "Queso", "Tomate"];

// Unir en un texto:
console.log(ingredientes.join(" + ")); // "Pan + Queso + Tomate"
console.log(ingredientes.join(", ")); // "Pan, Queso, Tomate"
```

## .slice(inicio, fin)

Extrae una porción del **array original** y te devuelve un **array nuevo**, sin tocar ni alterar el **original**. El índice de fin no se incluye en el resultado.

```
const frutas = ["Frutilla", "Banana", "Manzana", "Naranja", "Kiwi"];

// Extraer elementos de la posición 1 a la 3 (sin incluir la 3):
const seleccionadas = frutas.slice(1, 3);
console.log(seleccionadas); // ["Banana", "Manzana"]
console.log(frutas); // ["Frutilla", "Banana", "Manzana", "Naranja", "Kiwi"] (¡Intacto!)
```

## .concat(otroArray)

Une dos o más arrays en uno solo **completamente** nuevo. A diferencia del operador de propagación (...), este es un **método clásico directo**.

```
const alumnosMañana = ["Ana", "Pedro"];  
const alumnosTarde = ["Luis", "Clara"];  
  
// Fusionar arrays:  
const todosLosAlumnos = alumnosMañana.concat(alumnosTarde);  
console.log(todosLosAlumnos); // ["Ana", "Pedro", "Luis", "Clara"]
```

---

## Documentación Oficial (Referencias)

- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>